

Proyecto de curso: Programación Vectorial en Ensamblador x86-64 (NASM/Linux)

Comparación de rendimiento entre una solución escalar y una solución vectorizada

1. Objetivos de aprendizaje e introducción a las extensiones vectoriales

1.1 Objetivos de aprendizaje

Al finalizar el proyecto, el estudiante será capaz de:

- **OA1.** Explicar el modelo de ejecución SIMD (*Single Instruction, Multiple Data*) y contrastarlo con el modelo escalar tradicional (SISD).
- **OA2.** Describir la evolución histórica y las diferencias técnicas entre MMX, la familia SSE, AVX, AVX2 y AVX-512.
- **OA3.** Escribir, en NASM, funciones de ensamblador x86-64 que utilicen registros XMM/YMM e instrucciones empaquetadas (*packed*).
- **OA4.** Aplicar correctamente las reglas de alineación de memoria (16/32 bytes) y resolver el problema del **remanente** (elementos que no completan un vector) mediante un bucle de cierre (*tail loop*).
- **OA5.** Usar **GDB** para inspeccionar registros vectoriales y memoria, y así verificar la correctud de un kernel SIMD y depurar errores típicos (desalineación, indexación, corrupción de acumuladores).
- **OA6.** Medir y comparar cuantitativamente el rendimiento (tiempo de ejecución, *speedup*, IPC) entre una implementación escalar y una vectorizada, e interpretar los resultados a la luz de conceptos como ancho de banda de memoria y la Ley de Amdahl.
- **OA7.** Comunicar y defender oralmente, con dominio técnico, decisiones de diseño e implementación de bajo nivel.

1.2 Introducción a las extensiones vectoriales de x86

Las extensiones vectoriales permiten que una sola instrucción opere simultáneamente sobre varios datos empaquetados en un registro ancho (paradigma **SIMD**), en lugar de procesar un dato a la vez como en el modelo escalar clásico. Esto reduce el número de instrucciones ejecutadas y aprovecha mejor el paralelismo de datos disponible en problemas como procesamiento de señales, gráficos, álgebra lineal o estadística sobre grandes volúmenes de datos.

Extensión	Año aprox.	Ancho de registro	Registros	Tipos de dato soportados	Notas clave
MMX	1997	64 bits	MM0–MM7	Enteros empaquetados	Comparte estado físico con la FPU x87; obsoleta
SSE	1999	128 bits	XMM0–XMM15	Float simple precisión	Primeros registros vectoriales dedicados
SSE2	2001	128 bits	XMM0–XMM15	Enteros y doubles	Parte del ISA base de x86-64
SSE3 / SSSE3 / SSE4	2004–2006	128 bits	XMM0–XMM15	Enteros / floats	Añaden instrucciones horizontales y de empaquetado
AVX	2011	256 bits	YMM0–YMM15	Float simple/doble	Introduce codificación VEX de 3 operandos
AVX2	2013	256 bits	YMM0–YMM15	Enteros y floats	Extiende operaciones enteras a 256 bits
AVX-512	2016+	512 bits	ZMM0–ZMM31	Enteros / floats	Añade registros de máscara k0–k7

Conceptos que el estudiante debe dominar antes de programar:

- **Instrucciones escalares vs. empaquetadas:** `movss/addss/mulss` operan sobre **un solo** float en la parte baja del registro XMM; `movaps/addps/mulps` (SSE, 4 floats) y `vmovaps/vaddps/vmulps` (AVX/AVX2, 8 floats) operan sobre **varios** floats a la vez.
- **Alineación:** `movaps/vmovaps` exigen que la dirección de memoria esté alineada (16 y 32 bytes respectivamente) o generan *fault*; `movups/vmovups` no lo exigen pero pueden ser más lentas.
- **Reducciones horizontales:** operaciones como la suma total de un vector requieren combinar los "carriles" (*lanes*) del registro (p. ej. `vhaddps`, `shuffles` + sumas, o extracción de los 128 bits altos con `vextractf128`).
- **Remanente (tail handling):** si `N` no es múltiplo del ancho vectorial (8 para AVX2 con floats), los últimos elementos deben procesarse con un bucle escalar de cierre.
- **Verificación de soporte del procesador:** antes de ejecutar código AVX2 se debe confirmar el *flag* correspondiente con `lscpu | cat /proc/cpuinfo | grep flags`.

2. Descripción del problema

2.1 Planteamiento: "Normalizador estadístico vectorizado"

El estudiante debe construir un programa que, dado un arreglo grande de números en punto flotante de precisión simple (por ejemplo, muestras de una señal o sensor), calcule sus estadísticos descriptivos y produzca una versión **normalizada (z-score)** del arreglo.

Cada estudiante (o pareja) debe entregar **dos implementaciones funcionalmente equivalentes** del núcleo de cómputo:

1. **Versión escalar**, escrita en NASM usando únicamente instrucciones escalares (`movss`, `addss`, `subss`, `mulss`, `divss`, `sqrtps`, `comiss`, etc.), procesando un elemento a la vez.
2. **Versión vectorizada**, escrita en NASM usando AVX2 (`vmovaps/vmovups`, `vaddps`, `vmulps`, `vsubps`, `vdivps`, `vsqrtps`, `vmaxps`, `vminps`, reducción horizontal), procesando 8 floats por iteración, **con manejo explícito del remanente**.

Para mantener el foco pedagógico en la vectorización (y no en el manejo de syscalls), se recomienda la siguiente arquitectura híbrida:

- El **programa principal (driver)** — lectura de archivo, reserva de memoria alineada, medición de tiempo, impresión de resultados — se escribe en C.
- Los **kernels de cómputo** se escriben **enteramente en NASM** y se enlazan como funciones externas respetando la convención de llamada **System V AMD64 ABI**. Como mínimo se deben implementar:
 - `float sum_array(float* arr, int n);`
 - `void compute_stats(float* arr, int n, float* mean, float* var, float* min, float* max);`
 - `void normalize_array(float* in, float* out, int n, float mean, float stddev);`
- **No se permite** el uso de intrínsecos de C (`_mm256_...`) ni de flags de auto-vectorización del compilador (`-O3 -mavx2`, etc.) en el kernel: la vectorización debe ser manual y explícita en ensamblador.

2.2 Entradas y salidas

Entrada: archivo binario `input.dat` con el siguiente formato:

- 4 bytes: entero de 32 bits N (little endian) = cantidad de elementos.
- $N \times 4$ bytes: arreglo de floats de 32 bits (little endian).

Se debe generar el archivo con un script (Python/NumPy es aceptable, ya que es solo generación de datos de prueba, no parte del kernel evaluado) para al menos estos tamaños:

- Tamaños pequeños para verificación de correctud: $N = 0, 1, 7, 8, 15, 16, 1000$.
- Tamaños grandes para medición de rendimiento: $N = 10^3, 10^5, 10^6, 5 \times 10^7$.

Salida esperada:

- **Por consola:** N , suma, media (μ), varianza (σ^2), desviación estándar (σ), mínimo, máximo, y tiempo de ejecución del kernel (en ciclos de reloj y en milisegundos).
- **Archivo binario** `output_scalar.dat` / `output_vector.dat` con el arreglo normalizado: $y[i] = (x[i] - \mu) / \sigma$.

2.3 Especificaciones de la resolución esperada

- **Precisión:** todo el cómputo en `float32`. Se acepta una tolerancia relativa de 1×10^{-4} al comparar resultados entre la versión escalar y la vectorial, dado que el orden distinto de las sumas en la reducción produce diferencias de redondeo (esto **no** se considera un error si está dentro de tolerancia).
- **Casos borde obligatorios:** $N = 0$ (debe reportarse como error controlado, no división por cero), $N = 1$, N no múltiplo de 8 (verifica el manejo del remanente), arreglo con todos los valores iguales ($\sigma = 0$, debe evitarse división por cero), valores negativos y valores extremos.
- **Alineación:** la memoria de los arreglos debe reservarse alineada a 32 bytes en el driver C (`aligned_alloc 0 posix_memalign`) y el kernel vectorial debe usar instrucciones alineadas (`vmovaps`) en el camino principal.

2.4 Metodología de comparación

a) Comparación de correctud

1. Un script de referencia (Python + NumPy) calcula los mismos estadísticos y el arreglo normalizado esperado para cada archivo de entrada.
2. Los resultados de **ambas** versiones del estudiante se comparan automáticamente contra la referencia y entre sí, verificando que el error relativo esté dentro de la tolerancia definida.
3. Adicionalmente, para al menos un caso pequeño ($N = 16$), el estudiante debe usar **GDB** para verificar manualmente, en tiempo de depuración:
 - el contenido de un registro YMM inmediatamente después de una instrucción `vaddps` (p. ej. `info registers ymm0` o `print $ymm0.v8_float` según la versión de GDB disponible en el laboratorio);
 - el contenido de memoria del arreglo de salida en un punto de ruptura posterior al bucle de normalización (`x/8fw &output[0]`).

Esta sesión debe documentarse (transcripción o capturas) como evidencia en el informe.

Rol de GDB en este proyecto: es la herramienta de **verificación funcional e inspección de bajo nivel** (registros, memoria, control de flujo instrucción a instrucción con `stepi/nexti`), y también sirve para comparar de forma cualitativa cuántas iteraciones/instrucciones ejecuta cada versión del bucle. No es una herramienta de medición de tiempo de pared; para eso se usan los mecanismos de la sección (b).

b) Comparación de rendimiento

1. Medir el tiempo de ejecución **exclusivamente del kernel de cómputo** (no de la lectura de archivo) con `clock_gettime(CLOCK_MONOTONIC, ...)` en el driver C, alrededor de cada llamada a la función `NASM`.
2. Repetir cada medición **al menos 30 veces** por tamaño de N y reportar el promedio y la desviación estándar.
3. Repetir el experimento para los 4 tamaños grandes indicados en 2.2, para observar el comportamiento del *speedup* en distintas escalas (efecto de caché L1/L2/L3 vs. memoria principal).

4. Calcular $\text{Speedup} = \text{tiempo_escalar} / \text{tiempo_vectorial}$ por tamaño y graficar N (eje X, escala logarítmica) vs. Speedup (eje Y).
5. Ejecutar `perf stat -e cycles,instructions,cache-misses ./programa` sobre ambas versiones para reportar instrucciones por ciclo (IPC) y tasa de fallos de caché, y discutir en el informe si el cuello de botella observado es de cómputo o de ancho de banda de memoria (relevante para explicar por qué el *speedup* real casi nunca alcanza el 8x teórico de AVX2).

3. Rúbrica de evaluación

Criterio	Puntos
Funcionalidad y correctud — versión escalar	10
Funcionalidad y correctud — versión vectorial (incluye manejo del remanente)	15
Calidad y uso apropiado de las extensiones vectoriales (alineación, elección de instrucciones, reducción horizontal)	15
a) Diagrama de arquitectura de ambos programas	15
b) Informe técnico detallado	25
c) Entrevista de defensa	20 (criterio de veto — ver 3.c)
Total	100

3.a Diagrama detallado de la estructura de los dos programas (15 pts)

Se debe entregar **un diagrama** (archivo independiente, PDF o PNG, referenciado en el informe; herramienta libre: draw.io, Lucidchart, PlantUML, etc.) que cubra, **para ambas versiones (escalar y vectorial)**:

1. **Diagrama de bloques de la arquitectura de software:** módulos existentes (driver en C, `libstats_scalar.asm`, `libstats_vector.asm`), y cómo se comunican (llamadas de función respetando la ABI System V, qué registros se usan para pasar argumentos y cuáles debe preservar el callee).
2. **Diagrama de flujo de control** del bucle principal de cada kernel, mostrando explícitamente: inicialización del acumulador, condición del bucle vectorizado (iteraciones de 8 en 8), transición al bucle remanente (escalar), y condición de salida. Debe quedar visualmente clara la diferencia estructural entre el bucle escalar (un elemento por iteración) y el vectorial (ocho elementos por iteración + cola).
3. **Tabla de asignación de registros** usada en el kernel crítico (ejemplo de formato): qué contiene `ymm0`, `ymm1`, `rdi`, `rsi`, `rcx`, etc. en cada etapa del algoritmo.
4. **Diagrama de memoria:** disposición de los arreglos de entrada/salida, alineación de la dirección base (múltiplo de 32 bytes), y cómo avanza el puntero en cada iteración (offset de 4 bytes en la versión escalar vs. 32 bytes en la vectorial).

Desglose de puntos: claridad y completitud general (5) · representación correcta del flujo de control incluyendo el remanente (5) · tabla de registros y mapa de memoria (5).

3.b Informe detallado de la solución (25 pts)

El informe debe incluir, como mínimo:

1. Introducción y objetivos del trabajo (2 pts).
2. Descripción del entorno de pruebas: modelo de CPU y *flags* de soporte (salida de `lscpu` / `/proc/cpuinfo` confirmando AVX2), versión de NASM, GCC y GDB usadas (2 pts).
3. Explicación de la implementación escalar, con fragmentos de código comentados y justificación de decisiones (4 pts).
4. Explicación de la implementación vectorial, con fragmentos de código comentados, justificando: elección de AVX2 (frente a SSE o AVX-512), estrategia de reducción horizontal, y manejo del remanente (5 pts).
5. **Casos de prueba:** tabla con al menos los 6 casos borde de la sección 2.3, indicando entrada, salida esperada (referencia), salida obtenida por cada versión, y resultado (pasa/no pasa) (5 pts).
6. **Resultados de rendimiento:** tabla de tiempos promedio $\pm$ desviación estándar por tamaño y versión, gráfico de *speedup* vs. N , y resultados de `perf stat` (IPC, fallos de caché) con su interpretación (4 pts).
7. Evidencia de la sesión de GDB (transcripción o capturas) descrita en 2.4.a (2 pts).
8. Conclusiones, limitaciones observadas y trabajo futuro (1 pt).

3.c Entrevista corta de defensa (20 pts — criterio de veto)

Entrevista individual de 10 a 15 minutos, sin acceso a apuntes, en la que el estudiante debe defender **su propio** trabajo. Ejemplos de preguntas que se pueden formular:

- ¿Cuál es la diferencia entre `movaps` y `movups`? ¿Cuándo usaría cada una y por qué?
- ¿Por qué su reducción horizontal usa la técnica elegida (`vhaddps`, `shuffles`, o `vextractf128`)? ¿Qué alternativa existía?
- Muestre en vivo, usando GDB, el contenido de un registro YMM en una iteración específica del bucle vectorial.
- Si N no es múltiplo de 8, explique exactamente qué instrucciones procesan los elementos sobrantes.
- El *speedup* medido no llega a 8x aunque AVX2 procesa 8 floats por instrucción: explique por qué (ancho de banda de memoria, latencia de instrucciones, overhead de la reducción, Ley de Amdahl).
- Explique la convención de llamada usada entre C y NASM: qué registros transportan los argumentos y quién debe preservar qué registros.

Regla de calificación: si el estudiante **no puede explicar o defender** decisiones básicas de su propio código —evidencia de que no lo escribió o no lo comprende— **la nota total del proyecto será cero**, sin importar el puntaje obtenido en los demás criterios de la rúbrica. La misma consecuencia aplica si se detecta código copiado no atribuido. Si el estudiante defiende

correctamente su trabajo, los 20 puntos se asignan según la profundidad y precisión de las respuestas (comprensión superficial vs. profunda, capacidad de generalizar los conceptos a otros problemas).

4. Logística del proyecto

- **Formato de trabajo:** En parejas.
- **Fecha de inicio:**
- **Fecha de entrega:** 1 de Octubre (6 semanas)
- **Entregables:** repositorio (código NASM, driver en C, `Makefile`, script de generación de datos de prueba, script de verificación de referencia, archivo del diagrama, informe en PDF).
- **Entorno técnico:** Linux (Ubuntu 20.04+ recomendado), NASM ≥ 2.15 , GCC, GDB ≥ 10 , `perf` (paquete `linux-tools`).
- **Verificación previa del CPU:** los estudiantes deben confirmar soporte de AVX2 (`lscpu` o `/proc/cpuinfo`) antes de iniciar. Si la computadora no cuenta con AVX2, el proyecto puede adaptarse a SSE2 (128 bits / 4 floats) vs. escalar, conservando los mismos objetivos de aprendizaje.
- **Cronograma Sugerido (4 semanas):**
 - semana 1 — diseño y diagrama;
 - semana 2 — implementación y pruebas de la versión escalar;
 - semana 3 — implementación, pruebas y *benchmarking* de la versión vectorial;
 - semana 4 — redacción del informe y preparación de la entrevista.
- **Integridad académica:** el proyecto debe compilar con el `Makefile` entregado; no se permite el uso de intrínsecos de C ni de auto-vectorización del compilador en los kernels evaluados. Sobre el Uso de IA, no está prohibido usarla, en el tanto que el estudiante sea capaz de explicar y justificar cada línea de código y cada decisión tomada en su proyecto.